L2B-2. Triggers and Database-side Automation

Database Management Systems

Key problem

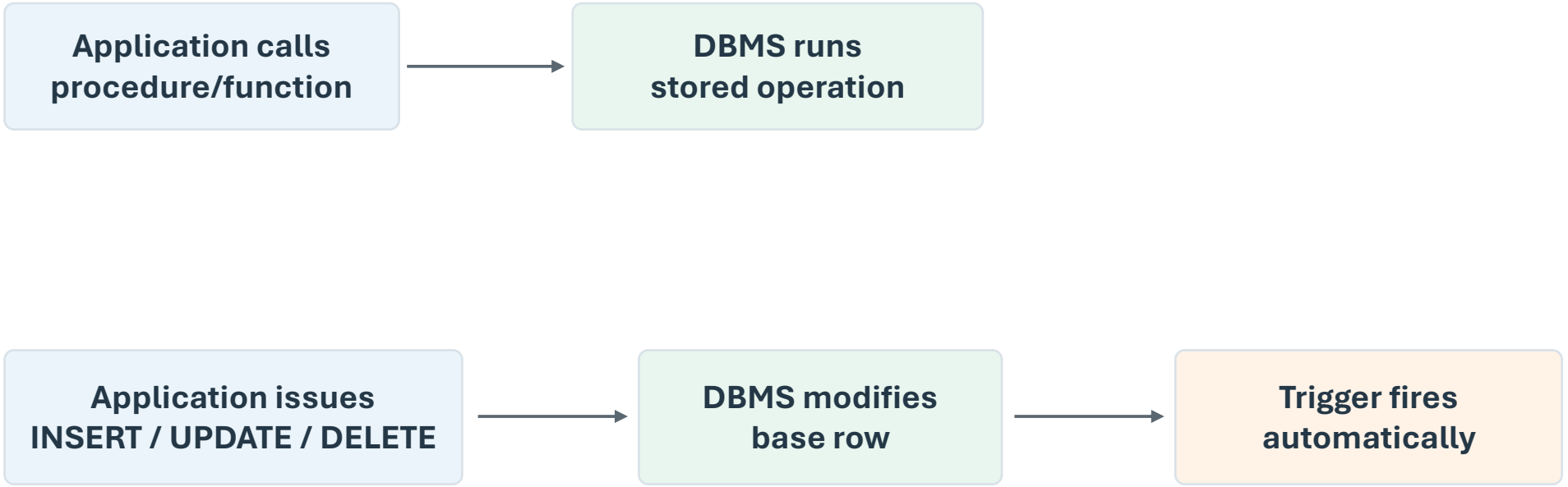
When should the DBMS automatically react to INSERT, UPDATE, or DELETE events?

Learning Outcomes

- Define a trigger as a stored database object activated by a data-change event.
  - Distinguish explicit routine calls from automatic trigger firing.
- Read a trigger by identifying timing, event, table, row context, condition, and action.
  - Use OLD and NEW correctly for INSERT, UPDATE, and DELETE.
- Create and test MySQL triggers using safe transaction-based experiments.
  - Inspect triggers using SHOW TRIGGERS, SHOW CREATE TRIGGER, and INFORMATION_SCHEMA.TRIGGERS.
- Decide when a rule belongs in a constraint, trigger, procedure, function, or application.

Connection With L2B-1

Procedures/functions are called explicitly; triggers fire automatically

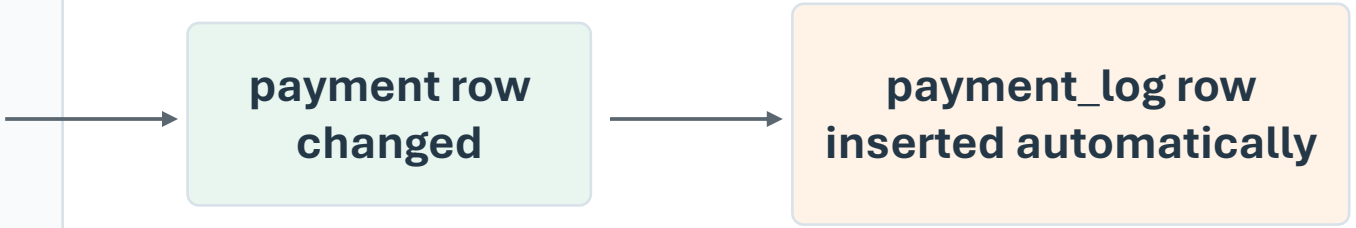


- L2B-1 focused on named operations that callers invoke intentionally.
- L2B-2 focuses on hidden database-side reactions attached to tables.
- **The design risk is higher: the application may not see that extra work is happening**.

Motivating Problem

Payment changes must be recorded even if the application forgets

```
UPDATE payment
SET amount = amount + 10
WHERE payment_id = 123;
```



- **Requirement**: every payment modification must leave an audit record.
- If auditing is only in application code, another application may bypass it.
- A trigger attaches the audit reaction to the table event itself.

Part I — Trigger Fundamentals

Definition, execution model, and design purpose.

Trigger Definition

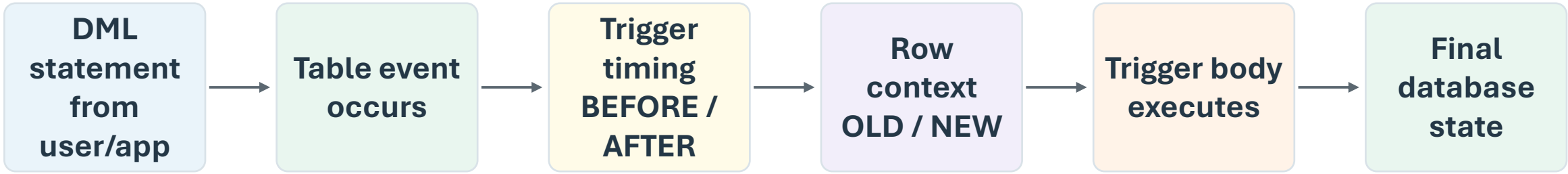
A trigger is a stored action activated by a database event



- Unlike a procedure, a trigger is not called with CALL.
- The DBMS fires it when the specified table event occurs.
- A trigger may check a rule, maintain derived data, or write audit/history information.
- Because execution is automatic, triggers must be documented and tested carefully.

Trigger Execution Pipeline

Read a trigger as a data-change reaction



- Do not start with syntax. First identify which event should cause a reaction.
- Then decide whether the action should occur before or after the row change.
- Finally, decide which old/new row values are needed by the action.

Anatomy of a Trigger

Component	Meaning	Question to ask
Trigger name	Schema object stored by the DBMS	What object will we inspect later?
Event	INSERT, UPDATE, or DELETE on a table	Which data-change operation fires it?
Timing	BEFORE or AFTER the row operation	Should action happen before validation/storage or after success?
Row context	OLD and/or NEW row values	Which row image is available?
Condition / body	SQL statements executed automatically	What rule or side effect is being implemented?

Identify these components before writing trigger SQL.

Events: INSERT, UPDATE, DELETE

A table format is the clearest way to compare row images

Trigger event	OLD row	NEW row	Typical question
INSERT	Not available	Inserted row	What values are entering the table?
UPDATE	Row before update	Row after update	What changed? Is the change allowed?
DELETE	Deleted row	Not available	What information must be archived before removal?

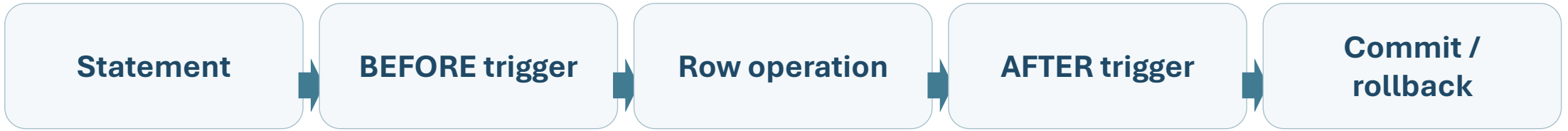
Use this table to predict which references are legal inside the trigger body.
Example: OLD.amount is meaningful in UPDATE/DELETE; NEW.amount is meaningful in INSERT/UPDATE.

Source: MySQL 8.4 Reference Manual §15.1.22 CREATE TRIGGER

Timing: BEFORE vs AFTER

Choose timing from semantics, not from habit

Timing	Good fit	Avoid when	Teaching example
BEFORE	Validate or normalize values before the row is stored	The action depends on successful final storage	Convert NULL/negative input into a controlled value
AFTER	Audit, maintain derived data, or update related records after success	The action needs to modify NEW values before storage	Write payment_log after payment is updated



OLD and NEW Row Context

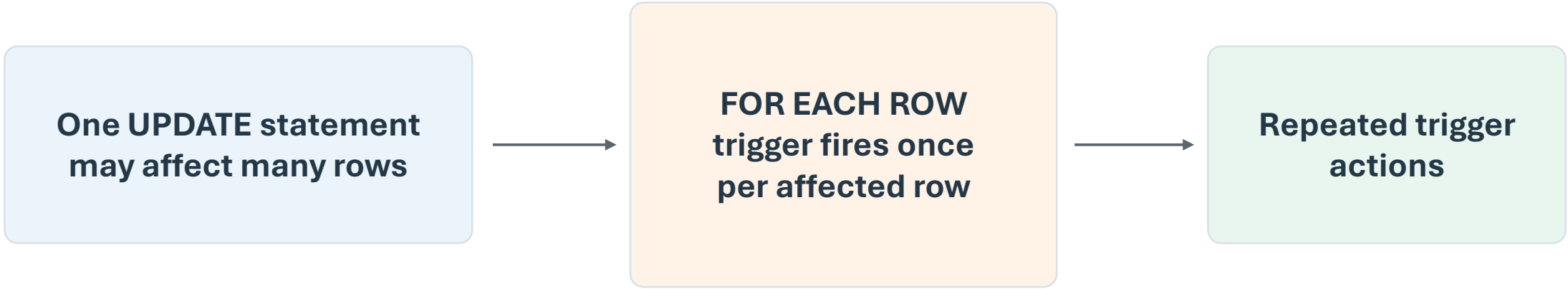
The row image available inside a row-level trigger

Event	OLD	NEW	Typical use
INSERT	—	new row	validate / log inserted values
UPDATE	row before update	row after update	compare old and new values
DELETE	deleted row	—	log or cleanup removed values

- In MySQL, use OLD.col_name and NEW.col_name inside the trigger body.
- A common bug is referencing OLD in an INSERT trigger or NEW in a DELETE trigger.
- For UPDATE, compare OLD and NEW to avoid unnecessary work.

Row-Level and Statement-Level Thinking

MySQL focus: FOR EACH ROW



- The same SQL statement can cause zero, one, or many trigger executions.
- When an UPDATE affects 500 rows, a row-level trigger body can execute 500 times.
- Performance and side effects must be evaluated per affected row, not only per statement.

Trigger Use Cases

Use a table only where it helps select the right tool

Use case	Why a trigger fits	Design caution
Audit trail	Every client-side change can be logged automatically	Audit table grows; log only useful fields
Derived data maintenance	Base update can immediately update redundant summary/state	Derived state may become inconsistent if trigger is wrong
Cross-client rule	Rule is enforced no matter which application writes data	Prefer CHECK/FK constraints when they express the rule
Auxiliary synchronization	Related table can be kept aligned with base table changes	Hidden side effects make debugging harder

Do not use a trigger just because SQL syntax allows it.
Use it when the data-change event itself is the correct boundary for the action.

When Not To Use A Trigger

A short decision checklist prevents overusing triggers

If the rule is...	Prefer...	Reason
A simple domain condition	CHECK constraint	Declarative, visible, optimized by DBMS
A parent-child existence rule	FOREIGN KEY	Standard referential integrity is clearer
A user workflow with choices	Stored procedure or application logic	Workflow should be explicit to the caller
A report or computation	View/function/query	Do not create hidden writes for read logic
A notification to external services	Application/event pipeline	Database triggers should not hide external side effects

Part II — MySQL Trigger Syntax and Inspection

Syntax details are introduced only after the execution model is clear.

CREATE TRIGGER Skeleton

Read syntax as a trigger contract

```
DELIMITER $$
CREATE TRIGGER
trigger_name
  trigger_time
trigger_event
  ON table_name
  FOR EACH ROW
BEGIN
  -- trigger action
END $$
DELIMITER ;
```

trigger_time
BEFORE / AFTER

trigger_event
INSERT / UPDATE / DELETE

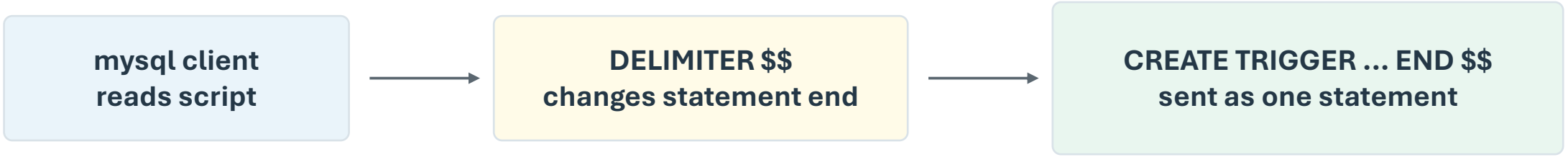
ON table_name
subject table

FOR EACH ROW
row-level trigger

BEGIN...END
SQL action body

DELIMITER In Trigger Scripts

The delimiter is a client-side instruction, not part of the trigger

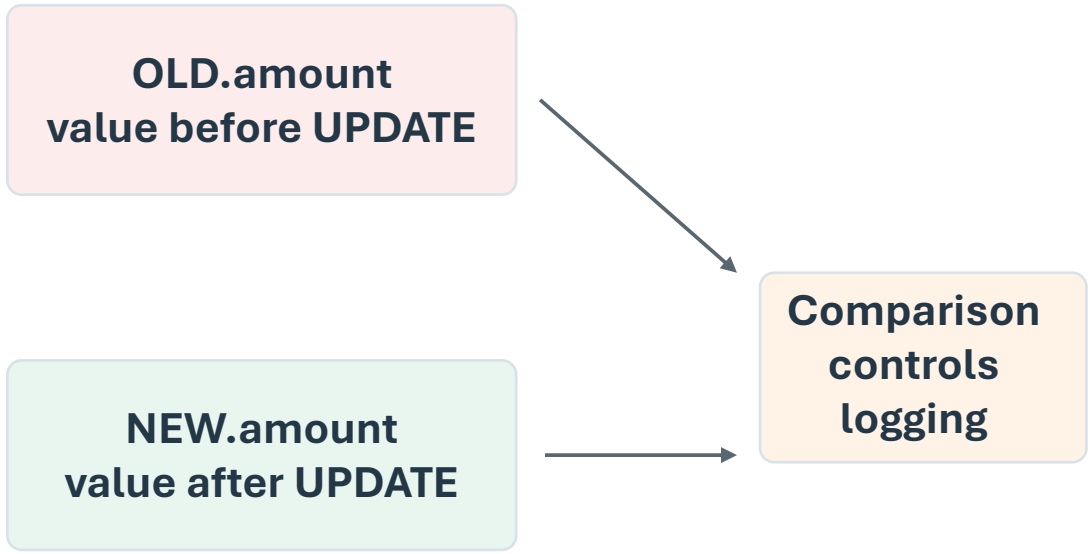


- A trigger body contains semicolons, so the client must not send the statement too early.
- DELIMITER $$ tells the client to wait until $$ before sending CREATE TRIGGER to the server.
- After the trigger is created, restore DELIMITER ; for normal SQL commands.
- SHOW CREATE TRIGGER will not show DELIMITER because it is not stored in the DBMS catalog.

Trigger Body Can Use OLD and NEW

Example: update changed amount only if it actually changed

```
IF OLD.amount <> NEW.amount THEN
  INSERT INTO payment_log(...)
  VALUES (OLD.payment_id,
          OLD.amount, NEW.amount, NOW());
END IF;
```



This pattern prevents logging when an UPDATE statement touches a row but does not actually change the audited value.

Inspecting Triggers In MySQL

Administrative evidence is part of the lab

Command / metadata query	What it reveals	Use in report
SHOW TRIGGERS;	Trigger name, event, timing, table, statement	List triggers created in schema
SHOW CREATE TRIGGER trg;	Exact stored trigger definition	Verify submitted SQL
INFORMATION_SCHEMA.TRIGGERS	Metadata such as event object, timing, statement	Compare trigger objects systematically
Test statement + SELECT log	Observed behavior after firing	Prove the trigger executed correctly

A trigger is not “done” when CREATE TRIGGER succeeds.
It must be inspectable, testable, and explainable.

Multiple Triggers and Ordering

MySQL can order triggers with the same event and timing




```
CREATE TRIGGER trigger_b
BEFORE UPDATE ON payment
FOR EACH ROW
FOLLOWS trigger_a
BEGIN ... END;
```

- By default, triggers with the same event and timing run in creation order.
- FOLLOWS or PRECEDES can make ordering explicit.
- Avoid trigger chains unless the ordering is documented and tested.

Restrictions and Safety Checks

Keep this as a reference table for implementation

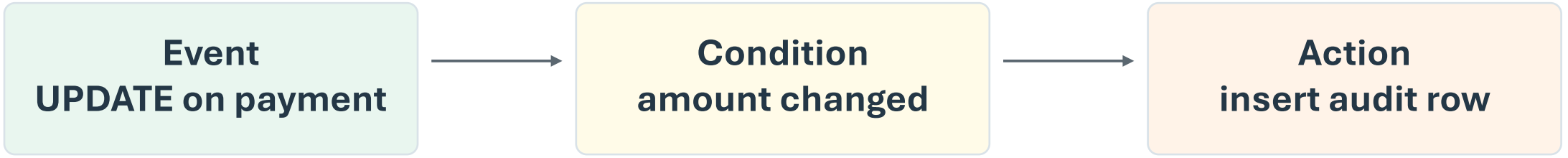
Issue	Consequence	What students should do
Trigger is attached to a table	MySQL triggers are not attached to views	Use view update mechanisms only in DBMSs that support them
Cannot rely on client-side DELIMITER inside object definition	DELIMITER will not appear in SHOW CREATE TRIGGER	Separate script syntax from stored definition
Hidden side effects	One UPDATE may write additional rows	Always test resulting table state
Multiple triggers with same timing/event	Execution order matters	Use FOLLOWS/PRECEDES only when necessary
Recursive/cascading logic	Hard to reason about correctness	Keep triggers small and documented

Part III — Guided SQL Demo: payment_log

A worked audit trigger with step-by-step testing.

Pattern 1: Audit Payment Changes

From requirement to trigger design



- Maintain **payment_log** when payment is inserted or changed.
- We first create the log table, then create triggers, then test with rollback.
- The important part is not only running SQL; students must explain event, timing, row context, and evidence.

Step 1 — Create an Audit Table

The log table is the explicit place where trigger side effects are stored

```
CREATE TABLE payment_log (
  log_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  payment_id SMALLINT UNSIGNED NOT NULL,
  old_amount DECIMAL(5,2),
  new_amount DECIMAL(5,2),
  action_type VARCHAR(10) NOT NULL,
  changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- The audit table should not be the same table that fires the trigger.
- Store enough data to explain what changed: key, old value, new value, action type, time.
- In a production system, also record user/session information if needed.

Step 2 — Create AFTER UPDATE Trigger

Use OLD and NEW to record before/after values

```
DELIMITER $$
CREATE TRIGGER payment_update_log
AFTER UPDATE ON payment
FOR EACH ROW
BEGIN
  IF OLD.amount <> NEW.amount THEN
    INSERT INTO payment_log
      (payment_id, old_amount, new_amount, action_type)
    VALUES
      (OLD.payment_id, OLD.amount, NEW.amount, 'UPDATE');
  END IF;
END $$
DELIMITER ;
```

- AFTER UPDATE: log only after the payment row has been successfully modified.
- FOR EACH ROW: the trigger can log one row per updated payment row.
- OLD and NEW provide the amount before and after the update.
- The IF condition avoids log noise when amount does not change.

Step 3 — Run A Safe Test

Use a transaction so lab data can be rolled back

```
START TRANSACTION;

SELECT payment_id, amount
FROM payment
WHERE payment_id = 1;

UPDATE payment
SET amount = amount + 1
WHERE payment_id = 1;

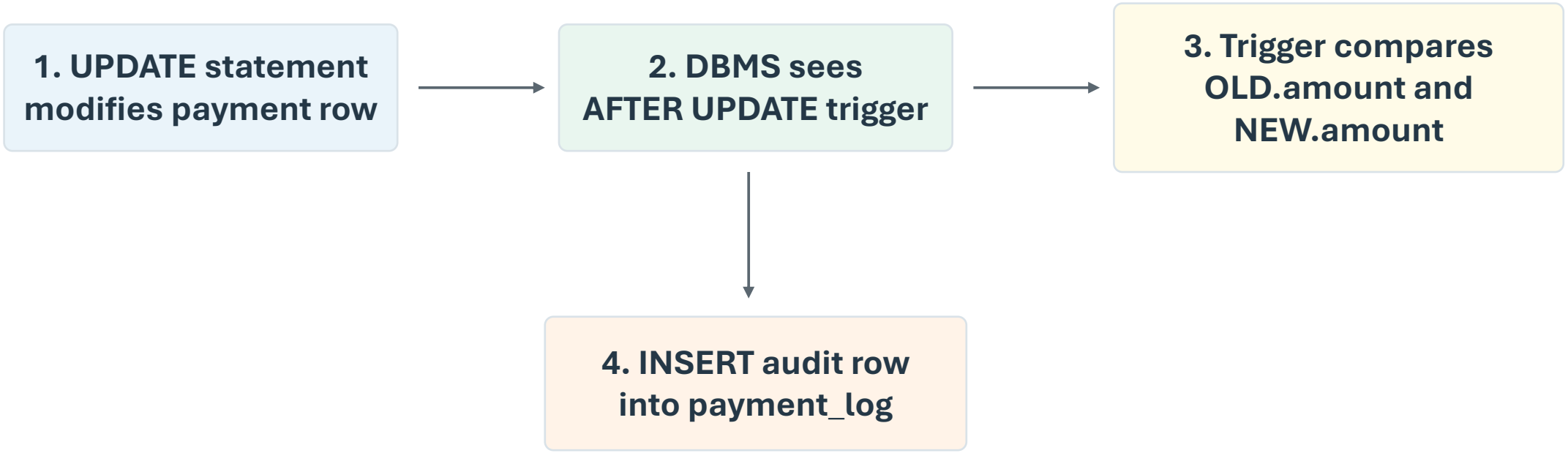
SELECT * FROM payment_log
WHERE payment_id = 1;

ROLLBACK;
```

- Predict first: **How many log rows should appear after the UPDATE?**
- Observe second: **inspect payment and payment_log after the UPDATE.**
- Rollback third: keep Sakila clean for the next run.
- Evidence: include trigger definition, test query, and log output.

Audit Trigger Execution Trace

What happens during the UPDATE?



Mini Exercise: Predict Trigger Output

Step	Statement	Expected payment_log row	Check after running
1	START TRANSACTION;	No log row yet	SELECT COUNT(*) FROM payment_log
2	UPDATE payment SET amount = amount + 1 WHERE payment_id = ?;	One row with OLD.amount and NEW.amount	SELECT * FROM payment_log ORDER BY changed_at DESC
3	ROLLBACK;	Changes rolled back if log table uses transactional engine	Verify payment and log state

Discussion: if the audit log is rolled back together with the data update, is that what you want? This depends on audit requirements.

Part IV —Sakila Example: upd_film

Example: upd_film

A trigger keeps film_text synchronized with film

```
CREATE TRIGGER upd_film
AFTER UPDATE ON film
FOR EACH ROW
BEGIN
  IF (OLD.title <> NEW.title)
     OR (OLD.description <> NEW.description) THEN
    UPDATE film_text
    SET title = NEW.title,
        description = NEW.description,
        film_id = NEW.film_id
    WHERE film_id = OLD.film_id;
  END IF;
END;
```

- **Event**: UPDATE on film.
- **Condition**: title or description changed.
- **Action**: update the auxiliary film_text row.

Step-by-Step: upd_film

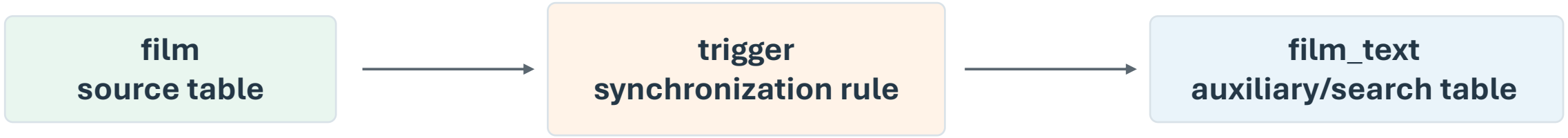
Analyze the trigger by event, condition, and action



- The auxiliary table stores data derived from film.
- The trigger reduces the risk that film_text becomes stale when film changes.
- The cost is hidden extra work on every relevant film update.

Design Analysis: Synchronizing Auxiliary Data

Ask whether stored redundancy is justified



- Good reason: the auxiliary table supports search or indexing features not stored the same way in film.
- Risk: if the trigger is disabled, broken, or bypassed, redundant data can diverge.
- Project question: Is the derived state necessary, or can it be computed from base tables when needed?

Test Plan For upd_film

A trigger example is incomplete without test evidence

```
START TRANSACTION;
SELECT title, description FROM film WHERE film_id = 1;
SELECT title, description FROM film_text WHERE film_id = 1;

UPDATE film
SET title = CONCAT(title, ' TEST')
WHERE film_id = 1;

SELECT title, description FROM film_text WHERE film_id = 1;
ROLLBACK;
```

- **Before**: record film and film_text values.
- **Action**: update film.title or film.description.
- **After**: verify film_text changed as expected.
- **Rollback**: keep the database reusable.

Pattern 2 — Specialized Integrity Enforcement

Maintain referential integrity using triggers.

```
create trigger timeslot_check1 after insert on section
referencing new row as nrow
for each row
when (nrow.time_slot_id not in (
        select time_slot_id
        from time_slot)) /* time_slot_id not present in time_slot */
begin
  rollback
end;

create trigger timeslot_check2 after delete on timeslot
referencing old row as orow
for each row
when (orow.time_slot_id not in (
        select time_slot_id
        from time_slot) /* last tuple for time_slot_id deleted from time_slot */
  and orow.time_slot_id in (
        select time_slot_id
        from section)) /* and time_slot_id still referenced from section*/
begin
  rollback
end;
```

Figure 5.9 Using triggers to maintain referential integrity.

- **Problem:** a data modification may create an invalid reference across relations.
- **Trigger role:** automatically check or repair affected rows after the triggering event.
- **Design question:** could a foreign-key constraint with ON DELETE / ON UPDATE express this rule more safely?

Pattern 3 — Maintain Derived State

Keep a derived attribute synchronized after grade changes.

```
create trigger credits_earned after update of takes on grade
referencing new row as nrow
referencing old row as orow
for each row
when nrow.grade <> 'F' and nrow.grade is not null
   and (orow.grade = 'F' or orow.grade is null)
begin atomic
   update student
   set tot_cred= tot_cred+
           (select credits
            from course
            where course.course_id= nrow.course_id)
   where student.id = nrow.id;
end;
```

Figure 5.10 Using a trigger to maintain *credits_earned* values.

Grade update → **Trigger fires** → **Credits updated** → **Derived state stays current**

- **Problem:**

tot_cred is derived from successful course completions but is stored in student.

- **Trigger role:**

keep redundant value consistent when the underlying grade changes.

- **Risk:** redundant data can drift if any relevant event is missed.

Core design trade-off:
compute on demand vs store and maintain.

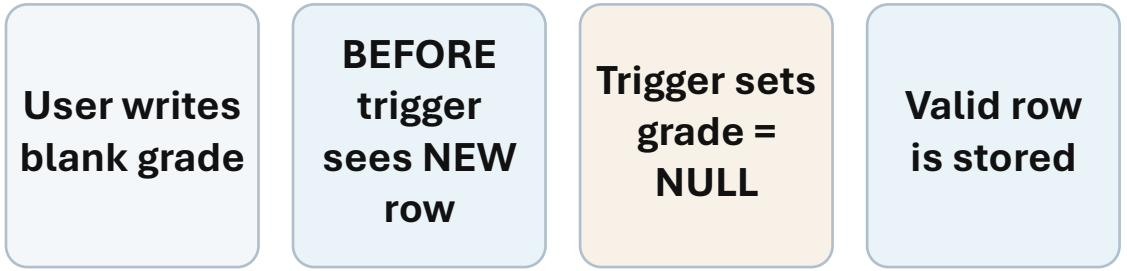
Pattern 4A — Transform Incoming Data Before Storage

a BEFORE trigger changes the row being written.

```
create trigger setnull before update of takes
referencing new row as nrow
for each row
when (nrow.grade = ' ')
begin atomic
   set nrow.grade = null;
end;
```

Figure 5.11 Example of using set to change an inserted value.

Before-trigger interpretation



- **Problem:**
application input uses a placeholder value that should not be stored directly.
- **Trigger role:**
normalize or repair the row before the database stores it.
- **Caution:**
do not hide important data-cleaning rules where application developers cannot see them.

Important distinction: BEGIN ... END groups trigger statements; it is not START TRANSACTION.

Pattern 4B — Threshold-Based Automatic Action

Reorder an item when inventory crosses below the minimum level.

```
create trigger reorder after update of level on inventory
referencing old row as orow, new row as nrow
for each row
when nrow.level <= (select level
                    from minlevel
                    where minlevel.item = orow.item)
and orow.level > (select level
                  from minlevel
                  where minlevel.item = orow.item)
begin atomic
    insert into orders
        (select item, amount
         from reorder
         where reorder.item = orow.item);
end;
```

Figure 5.12 Example of trigger for reordering an item.

old level above minimum

UPDATE level

new level below minimum

insert reorder request

What is new in this example?

- The trigger does not fire for every low inventory value in the same way.
- It checks a transition: old level was above the minimum, new level is at or below the minimum.
- The action inserts an order only when the threshold is crossed.
- This avoids repeated reorder actions while the item remains below the minimum.

Design point: many useful triggers depend on a transition, not only on the new value.

Synthesis: When Should a Trigger Be Used?

Use the examples as design patterns, then choose deliberately.

Can a declarative constraint express it?

Can a procedure/application workflow express it better?

Must it react automatically to data change?

Is the side effect small and testable?

Good trigger candidates

- Audit records for data changes.
- Derived state that must be maintained automatically.
- Small integrity actions that are tied directly to one DML event.
- Input normalization before storage when the rule is simple and stable.

Avoid or reconsider triggers when

- A CHECK, FOREIGN KEY, or generated column is sufficient.
- The action is a complex business workflow.
- The trigger would start chains of hidden side effects.
- Testing, debugging, or portability becomes difficult.

Mini Exercise: Classify the Trigger Pattern

For each case, answer:

- What is the triggering event?
- Which pattern does it match?
- Is there a better alternative than a trigger?
- What evidence would you show in MySQL?

A payment amount changes; store old_amount and new_amount in payment_log.	**Audit / history**
A blank grade is submitted; store NULL instead of an empty string.	**Transform before storage**
A student grade changes from F to B; update total earned credits.	**Maintain derived state**
Inventory level crosses below the minimum; insert a reorder request.	**Threshold-based action**
Deleting a parent row should remove or repair dependent rows.	**Integrity enforcement**

Part VI — Lab L2B-2: Triggers in MySQL

The lab asks for evidence and explanation, not only a final script.

Lab Setup and Safety

Run trigger experiments without permanently damaging Sakila

MySQL 8.x + Sakila

Save script L2B2_triggers_<id>.sql

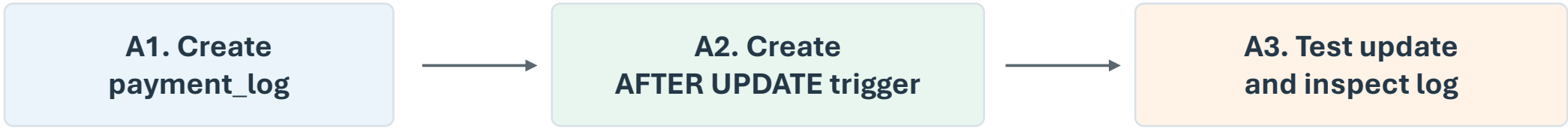
Use transactions for data-change tests

- Use SHOW CREATE TRIGGER and INFORMATION_SCHEMA.TRIGGERS as evidence.
- When testing DML, start with START TRANSACTION and end with ROLLBACK unless instructed otherwise.
- Drop lab-created triggers and log tables at the end if using a shared database.

Lab Task A — payment_log Audit

LAB

Create the audit table and one trigger first



- **Minimum evidence:** CREATE TABLE, CREATE TRIGGER, test UPDATE, SELECT payment_log, SHOW CREATE TRIGGER.
- **Required explanation**: event, timing, OLD/NEW values, and why AFTER is used.
- **Extension**: add an AFTER INSERT trigger to log new payments.

Lab Task B — Test All Trigger Paths

Use a test matrix instead of informal screenshots

Path	Action	Expected trigger behavior	Evidence to submit
A	INSERT new payment	INSERT trigger writes one log row if implemented	Before/after SELECT on payment_log
B	UPDATE payment amount	UPDATE trigger records old and new amount	Log row with changed values
C	UPDATE unrelated column	Either no log row or log row by design	Explain chosen design
D	ROLLBACK test transaction	Payment and log state should match transactional expectation	Final SELECT after ROLLBACK

Source: UET Lab 04 Trigger/View

Lab Task C — Trigger Metadata

Use DBMS inspection, not only screenshots of result rows

```
SHOW TRIGGERS FROM sakila
LIKE 'payment';

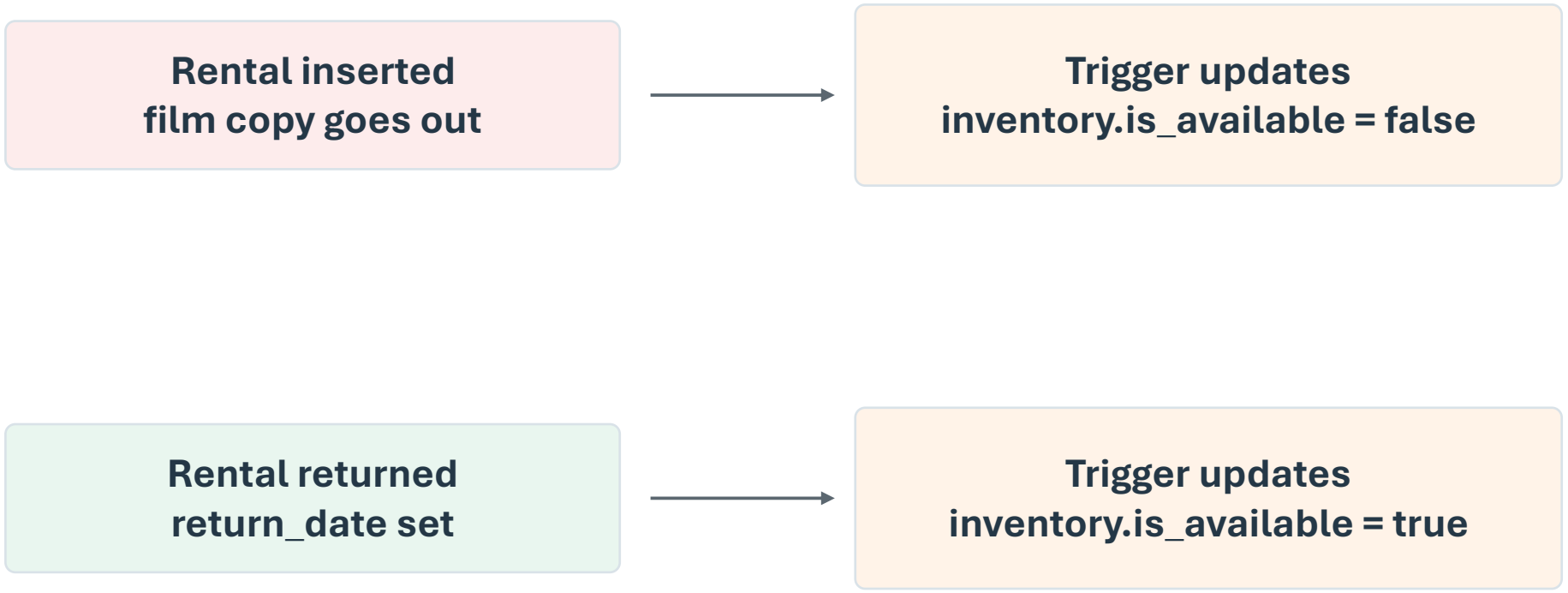
SHOW CREATE TRIGGER
payment_update_log;

SELECT trigger_name,
event_manipulation,
       action_timing,
action_statement
FROM information_schema.triggers
WHERE trigger_schema = 'sakila';
```

- Identify the event and timing from metadata.
- Confirm the trigger belongs to the expected table.
- Check the action statement stored by the DBMS.
- This is the administrative view of the trigger.

Lab Task D — inventory.is_available

Maintain availability state from rental events



Availability Design Question

Stored state vs derived state

Derived from rental table
query current unreturned rental



Stored in inventory.is_available
maintained by triggers

- Derived state avoids redundancy but may require repeated joins/filters.
- Stored state can make reads simple but must be kept correct on every rental/return event.
- A trigger is justified only if the read benefit outweighs maintenance risk and complexity.

Availability Trigger 1 — Rental Insert

A rental creates an unavailable inventory item

```
DELIMITER $$
CREATE TRIGGER rental_after_insert
AFTER INSERT ON rental
FOR EACH ROW
BEGIN
  UPDATE inventory
  SET is_available = FALSE
  WHERE inventory_id =
NEW.inventory_id;
END $$
DELIMITER ;
```

- Event: INSERT on rental.
- Row context: NEW.inventory_id identifies the rented copy.
- Action: update the related inventory row.
- Test: insert a rental row in a transaction, check inventory, then rollback.

Availability Trigger 2 — Return Update

A return can make an inventory item available again

```
DELIMITER $$
CREATE TRIGGER rental_after_update
AFTER UPDATE ON rental
FOR EACH ROW
BEGIN
  IF OLD.return_date IS NULL
     AND NEW.return_date IS NOT NULL
THEN
    UPDATE inventory
    SET is_available = TRUE
    WHERE inventory_id =
NEW.inventory_id;
  END IF;
END $$
DELIMITER ;
```

- Condition detects the transition: not returned → returned.
- This avoids setting availability for unrelated rental updates.
- Before using this in a real project, consider multiple copies, failed updates, and concurrent transactions.

Lab Task E — Test Availability Carefully

Write expected states before executing SQL

```
START TRANSACTION;

-- choose an inventory_id and insert a rental
INSERT INTO rental(rental_date, inventory_id,
customer_id, staff_id)
VALUES (NOW(), @inventory_id, @customer_id, 1);
SELECT is_available FROM inventory WHERE
inventory_id = @inventory_id;

-- simulate return
UPDATE rental SET return_date = NOW()
WHERE rental_id = LAST_INSERT_ID();
SELECT is_available FROM inventory WHERE
inventory_id = @inventory_id;

ROLLBACK;
```

- Expected after insert: is_available = FALSE.
- Expected after return update: is_available = TRUE.
- If results differ, inspect trigger definitions and event timing.
- Do not skip rollback in shared Sakila.

Lab Submission Checklist

Evidence must show execution and explanation

Deliverable	Minimum evidence	Why it matters
SQL script	CREATE TABLE, CREATE TRIGGER, test statements, cleanup	Reproducibility
Trigger metadata	SHOW TRIGGERS or SHOW CREATE TRIGGER	Confirms stored object
Before/after output	SELECT from base table and log/derived table	Shows behavior, not only syntax
Short explanation	Event, timing, OLD/NEW, side effect	Connects lab to theory
Design note	Why trigger instead of constraint/procedure/application	Project-level reasoning

Part VII — Decision, Project, Homework

Use triggers as a design choice, not as default database programming.

Trigger or Something Else?

Use this decision matrix before choosing a trigger

Requirement type	Better choice	Reason
Expose a reusable logical relation	View	Read interface; no hidden write
Run an explicit multi-step operation	Procedure	Caller intentionally invokes operation
Compute a scalar reusable value	Function	Can be composed in SQL expressions
Ensure simple data validity	Constraint	Declarative and visible to optimizer/users
React automatically to a table change	Trigger	Event itself should cause the action
Notify external system	Application/event service	Avoid hidden external side effects inside DBMS

Project Connection

Specify one automatic database reaction in your project

1. What table event would fire it?

2. Should it be BEFORE or AFTER? Why?

3. Which OLD/NEW values are needed?

4. What table or state will the trigger modify?

5. What is the alternative design without a trigger?

6. How will you test rollback and failure behavior?

Homework B2 — Required Tasks

Modify examples and explain the design choice

Task	Requirement	Evidence
HW1	Modify payment_log to capture both INSERT and UPDATE on payment.	SQL script, trigger metadata, two test cases
HW2	Extend upd_film so changes to description are reflected in film_text.	Modified trigger and before/after SELECT
HW3	Design one trigger-like rule from your project, then decide whether it should be a trigger.	Event-condition-action description and alternative design
HW4	Answer: why can a trigger make debugging harder than a stored procedure?	Short explanation using one lab example

Homework B2 — Extension Options

Choose one if your project needs deeper trigger logic

- **Extension A.** Add a BEFORE trigger that normalizes or rejects an invalid value; explain why a CHECK constraint is not enough.
- **Extension B**. Implement a trigger-maintained summary table and compare it with a regular view query.
- **Extension C**. Create two triggers with the same event/timing and use FOLLOWS or PRECEDES; document the observed order.
- **Extension D**. Port one trigger idea to PostgreSQL, Oracle, or SQL Server and compare OLD/NEW style variables.

Part VIII — Beyond the Core

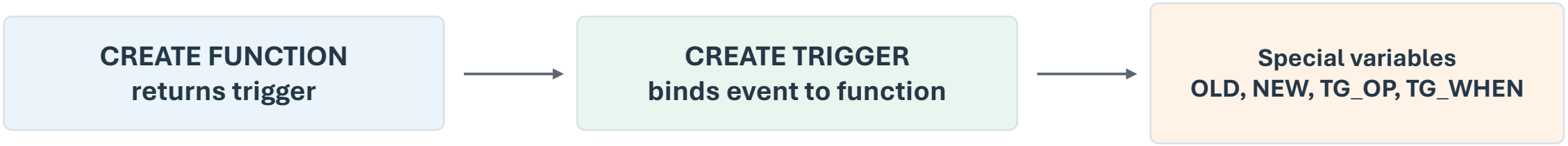
Other DBMSs implement data-change reactions differently.

Sources: official PostgreSQL, Oracle, Microsoft SQL Server, and MongoDB documentation.

Beyond MySQL: PostgreSQL Triggers

ADVANCED

Trigger functions separate the function from the trigger binding



- PostgreSQL trigger logic is commonly written as a trigger function.
- The trigger function can inspect special variables such as OLD, NEW, TG_OP, and TG_WHEN.
- Project exploration: reimplement payment_log in PostgreSQL and compare the two-step function/trigger model.

Official docs: PostgreSQL 18, Chapter 37 Triggers; PL/pgSQL Trigger Functions.

Beyond MySQL: Oracle Triggers

ADVANCED

Oracle adds INSTEAD OF and compound trigger patterns

Row-level DML triggers
OLD / NEW pseudorecords

INSTEAD OF triggers
define DML on views

Compound triggers
multiple timing sections

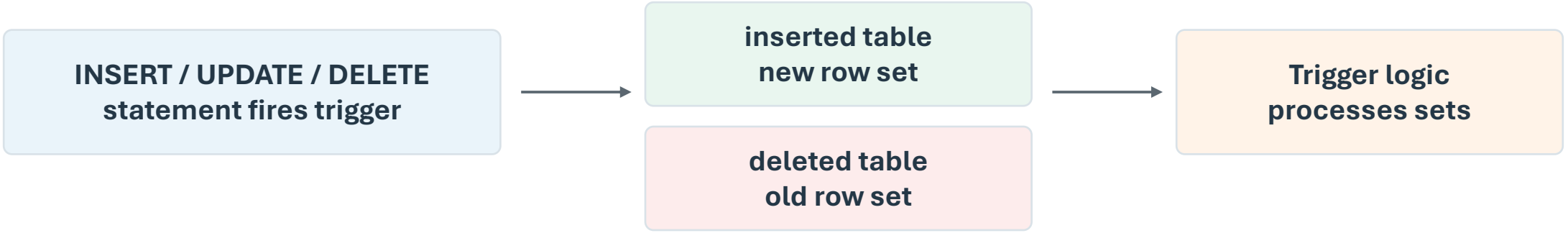
- Oracle can use INSTEAD OF triggers to define behavior for complex view updates.
- Compound triggers can share state across timing points in a single trigger body.
- Project exploration: compare MySQL's base-table trigger model with Oracle triggers on views.

Official docs: Oracle Database PL/SQL Language Reference, PL/SQL Triggers.

Beyond MySQL: SQL Server DML Triggers

ADVANCED

SQL Server exposes changed rows through inserted and deleted tables



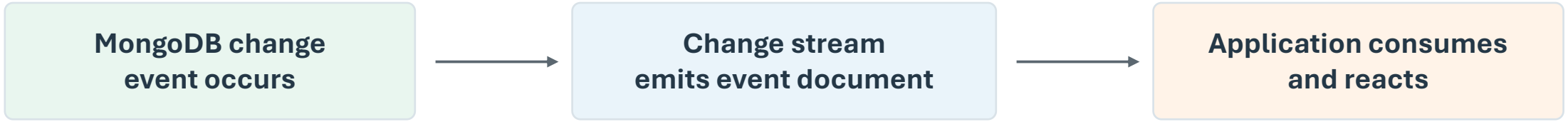
- SQL Server trigger logic should be written for sets, not single rows.
- The inserted/deleted logical tables can contain multiple rows affected by one statement.
- Project exploration: compare row-by-row MySQL thinking with SQL Server set-based trigger thinking.

Official docs: Microsoft Learn, CREATE TRIGGER (Transact-SQL), DML triggers.

Beyond MySQL: MongoDB Change Streams

ADVANCED

Not a trigger, but a database-change reaction mechanism



- Change streams are consumed by applications; they are not the same as MySQL triggers executed inside the database statement.
- They are useful for event-driven architectures, notifications, and external workflows.
- Project exploration: compare payment_log trigger with a service consuming a MongoDB change stream.

Official docs: MongoDB Manual, Change Streams.

Further Reading and Resources

Use official documentation for project extensions

Purpose	Resource
Required textbook reading	DSC 7e Ch.5 §5.3 Triggers, pp.206–213.
MySQL implementation	MySQL 8.4 Manual §15.1.22 CREATE TRIGGER; §27.3 Using Triggers; §27.8 Restrictions.
PostgreSQL extension	PostgreSQL 18 docs: Chapter 37 Triggers; §41.10 Trigger Functions.
Oracle extension	Oracle Database docs: About Triggers; PL/SQL Language Reference for compound and INSTEAD OF triggers.
SQL Server extension	Microsoft Learn: CREATE TRIGGER and DML triggers with inserted/deleted tables.
MongoDB comparison	MongoDB Manual: Change Streams as application-visible data-change events, not SQL triggers.

References are official documentation pages; use latest version relevant to your project DBMS.